Code-Layout, Readability and Reusability

Date	06 July 2026
Team ID	xxxxxx
Project Name	Smart Lender
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP-8 standards followed with 4-space indentation across all Python files
2	Proper File Structure	Files and folders are logically organized	Yes	Separate folders for data/, model/, templates/, static/, notebooks/, and utils/
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables like applicant_income, loan_amount, credit_history used instead of generic names
4	Function / Method Names	Functions are descriptively named	Yes	Functions such as preprocess_data(), train_models(), predict_loan_status() clearly indicate purpose
5	Code Comments	Inline and block comments explain logic	Partial	Docstrings added for all functions; inline comments present for complex encoding logic
6	Modular Design	Code is split into reusable functions/modules	Yes	Separate modules for data loading, preprocessing, model training, and Flask routes

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
7	No Redundant Code	Duplicate or unused code is removed	Yes	Helper functions created to avoid repetition in data encoding and model evaluation
8	Error Handling	Exceptions and errors are handled gracefully	Partial	Try-except blocks added in Flask routes; model training errors logged but not fully handled

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	data_loader.py	Python / Pandas	Used in EDA notebook, model training script, and Flask app	High
2	preprocessor.py	Python / Scikit-learn	Reused across data cleaning, feature encoding, and prediction pipeline	High
3	model_trainer.py	Python / Scikit-learn, XGBoost	Used for training and comparing Decision Tree, Random Forest, KNN, and XGBoost	Medium
4	evaluation_utils.py	Python / Matplotlib, Seaborn	Reused in model comparison plots, EDA notebook, and report generation	Medium
5	loan_predictor.py	Python / Flask	Core prediction logic reused in the API endpoint and batch evaluation script	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Well-organized folder hierarchy with clear separation of concerns; follows PEP-8 guidelines
Readability	4	Meaningful variable names, descriptive function names, and proper docstrings make the code easy to understand
Reusability	4	Core modules like data_loader.py and preprocessor.py are designed to be reused across multiple components
Documentation / Comments	3	Docstrings are present for all major functions, but some inline comments could be improved for the model comparison logic
Overall Score	3.75 / 5	The codebase demonstrates good engineering practices with modular design and clear naming. Minor improvements needed in error handling and inline documentation.